Reviewer Pack — Minimal Rank of Quantum Entanglement Information over AC0 Pa…

Entry c32bd38878d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 06:26:07 UTC

Minimal Rank of Quantum Entanglement Information over AC0 Parity Circuit Threshold

Entry ID: `c32bd38878d4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 06:26:07 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Entanglement) **Field B** (complexity object): Complexity Theory: AC0 Parity Circuit Complexity

Statement:

> ['For any AC0 parity circuit C computing a function f with threshold θ, the quantum entanglement information I(C) is upper-bounded by 2θ log(2), where I(C) is computed as the minimum average entanglement across all possible bipartitions of the input space.', "There exists an absolute constant c such that for any AC0 parity circuit C computing a function f with threshold θ ≤ c, there is an equivalent circuit C' with at most 2^θ(1 + log(θ)) gates.", 'For any AC0 parity circuit C, if the quantum entanglement information I(C) exceeds 2θ log(2), then there exists an input configuration x such that the output of f(x) is not determined by a local check on the computation of C.']

Rationale (proposer's reasoning):

> ["Quantum entanglement information measures the non-local correlations in a quantum system. If this information can be bounded by a function of the AC0 parity circuit's threshold, it may provide a new way to understand the complexity of certain functions.", 'The proposed relationship suggests that the entanglement information could serve as a resource for constructing more efficient circuits or understanding the inherent complexity of computations.', 'Exploring the connection between quantum entanglement and computational complexity can potentially lead to new insights into both fields.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `dbf2645a423534b5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Quantum entanglement information I(C) for an AC0 parity circuit C with threshold θ exceeds 2θ log(2), where I(C) is computed as the minimum average entanglement across all bipartitions.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define constants and parameters
    n = 10  # Example size, can be adjusted
    theta = random.randint(1, 5)  # Threshold for AC0 parity circuit

    # Generate a random AC0 parity circuit (simplified example)
    circuit = [random.choice([0, 1]) for _ in range(n)]

    # Compute quantum entanglement information (simplified example)
    I_C = sum(circuit) / n * math.log2(2)

    # Check if the conjecture holds
    upper_bound = 2 * theta * math.log2(2)
    conjecture_holds = I_C <= upper_bound

    return {
        "metric_name": "Quantum Entanglement Information",
        "metric_value": I_C,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Threshold {theta}, Circuit {circuit}"
    }

if __name__ == "__main__":
```

```python
import sys
seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, **result}}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = results[first_failing_seed]["counterexample"]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {"seed": 11, **result}
TRIAL: {"seed": 23, **result}
TRIAL: {"seed": 37, **result}
TRIAL: {"seed": 53, **result}
TRIAL: {"seed": 71, **result}
TRIAL: {"seed": 89, **result}
TRIAL: {"seed": 103, **result}
TRIAL: {"seed": 127, **result}
TRIAL: {"seed": 149, **result}
TRIAL: {"seed": 167, **result}
TRIAL: {"seed": 191, **result}
TRIAL: {"seed": 211, **result}
TRIAL: {"seed": 233, **result}
TRIAL: {"seed": 257, **result}
TRIAL: {"seed": 277, **result}
TRIAL: {"seed": 311, **result}
TRIAL: {"seed": 347, **result}
TRIAL: {"seed": 389, **result}
TRIAL: {"seed": 421, **result}
TRIAL: {"seed": 463, **result}
TRIAL: {"seed": 503, **result}
TRIAL: {"seed": 547, **result}
TRIAL: {"seed": 593, **result}
TRIAL: {"seed": 631, **result}
TRIAL: {"seed": 677, **result}
TRIAL: {"seed": 727, **result}
TRIAL: {"seed": 773, **result}
TRIAL: {"seed": 821, **result}
```

```
TRIAL: {"seed": 877, **result}
TRIAL: {"seed": 929, **result}
RESULT: SUPPORTED mean=0.5066666666666666 std=0.1590248058916875 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The empirical test only covers a small sample size of n ≤ 15, which may not be sufficient to validate the conjecture over a wide range of circuit sizes and thresholds. The metric could scale trivially with n, meaning that the observed results might not hold for larger circuits.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test only covers a small sample size of n ≤ 15, which may not be sufficient to validate the conjecture over a wide range of circuit sizes and thresholds. The observed results might not hold for larger circuits. | next: Increase the sample size significantly to cover a broader range of circuit sizes and thresholds, ensuring that the test is more representative of the conjecture's validity.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14329
2	propose	ollama_remote	glm4:latest	0	0	14336
3	preregistration	ollama_remote	glm4:latest	0	0	9737
4	novelty	ollama_remote	glm4:latest	0	0	8830
5	novelty	ollama_remote	glm4:latest	0	0	8203
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9982
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	5899
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6401
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6505
10	critic	ollama_remote	glm4:latest	0	0	12913
11	judge	ollama_remote	glm4:latest	0	0	12896

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 110032 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/c32bd38878d4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c32bd38878d4.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c32bd38878d4.tar.gz` (if generated)